

UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



FACULTAD DE INGENIERIA

UNIVERSIDAD NACIONAL DE CUYO

FACULTAD DE INGENIERÍA

LICENCIATURA EN CIENCIAS DE LA COMPUTACIÓN

Sistema de Wave Picking para la Preparación de Pedidos en un Almacén

Informe Técnico y Análisis Algorítmico

Integrantes del Equipo:
Funes Matias
Flores Cesar
Cornejo Jonathan
Forconesi Julieta

Materia: Algoritmos y Estructuras de Datos II
Profesor: Carlos Catania , Jorge Guerra
Fecha de Entrega: 09/11/2026

UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO

FACULTAD DE INGENIERIA

1.Introducción:

El presente informe detalla las decisiones de diseño, estructuras de datos y algoritmos implementados en el simulador de Wave Picking. El objetivo principal del proyecto fue modelar un almacén (Warehouse) y optimizar la preparación de pedidos mediante la agrupación eficiente de tareas (Waves) y la búsqueda de rutas de recolección, respetando las restricciones de tráfico y capacidad física exigidas por la cátedra.

2. Modelado del Entorno y Estructuras de Datos

El almacén fue modelado como un espacio cartesiano discretizado en una matriz bidimensional (grilla) de tamaño ancho_x por largo_y. Cada unidad mínima es una instancia de la clase Cell, que almacena su tipo (Rack, Pasillo, Puerta, Despacho) y, crucialmente, el sentido de circulación (N, S, E, O, AMBOS) si se trata de un pasillo.
Se eligió esta representación geométrica porque permite validar solapamientos y adyacencias en tiempo constante O(1). Para el cálculo de rutas, esta grilla se transforma en un Grafo Dirigido donde los nodos son las celdas transitables y las aristas representan el movimiento permitido. Se implementó el grafo utilizando listas de adyacencia (un diccionario de diccionarios) en lugar de una matriz de adyacencia, ya que el almacén es un grafo disperso (cada celda se conecta como máximo a 4 vecinos). Esto reduce la complejidad espacial a O(V + E), permitiendo escalar a almacenes de gran tamaño sin agotar la memoria.
Para la gestión del stock, se implementó una HashTable con encadenamiento (evitando los diccionarios nativos de Python). Las claves son los IDs de los productos, logrando búsquedas, inserciones y actualizaciones en tiempo promedio O(1).

3. Generación y Asignación de Waves

Para decidir qué pedidos pertenecen a una misma wave, se implementó un algoritmo Greedy (Voraz) determinista. El proceso consiste en:

1. Ordenar los pedidos pendientes según su prioridad (de mayor a menor).
2. Iterar sobre los pedidos, buscando a los pickers que tengan la capacidad física suficiente (capacidad_carga_kg) para transportar el peso del pedido.
3. Entre los pickers viables, asignar el pedido a aquel que tenga la menor carga acumulada (balanceo de carga).
4. Si la wave actual de ese picker alcanza su límite de pedidos o productos, el sistema crea automáticamente una nueva wave (ej. WAVE_002) para ese mismo picker.

Justificación: El problema de agrupar pedidos bajo restricciones de capacidad es una variante del Bin Packing, el cual es un problema NP-Hard. Dado que el PDF prohíbe el uso de metaheurísticas (algoritmos genéticos, simulated annealing, etc.), el enfoque Greedy es la solución algorítmica más eficiente y justificable. Garantiza un tiempo de ejecución polinómico O(P log P) y asegura una distribución equitativa del trabajo, evitando que un picker quede sobrecargado mientras otros están inactivos.

4. Algoritmos de Enrutamiento y Búsqueda de Caminos

Para determinar la distancia entre las ubicaciones y trazar el recorrido respetando los sentidos únicos de los pasillos, se utilizó el Algoritmo de Dijkstra.
Justificación: Se descartaron alternativas como BFS (porque no maneja pesos ni restricciones complejas de dirección de forma nativa) o A* (porque requiere una heurística que suele subestimar el camino real cuando hay pasillos de un solo sentido, perdiendo eficiencia). Dijkstra garantiza el camino más corto real en grafos con pesos positivos.
Implementación y Optimización:
Para cumplir con los requisitos de la materia, no se utilizaron librerías externas como heapq. Se implementó una Priority Queue (Min-Heap) propia y la mencionada HashTable.
Además, se incorporó un Caché de Distancias (Memoización): cada vez que se calcula la ruta desde un origen, los resultados se guardan en una HashTable. Si un picker vuelve a consultar una ruta desde el mismo nodo, el sistema la devuelve en O(1), en lugar de recalcular Dijkstra O((V+E) log V). Para obtener la ubicación física, el sistema busca la celda de pasillo adyacente a la celda del rack donde se encuentra el producto, ya que el picker no puede atravesar las estanterías.

5. Análisis de Complejidad y Comportamiento

El análisis de un sistema algorítmico no solo requiere medir su velocidad en condiciones ideales, sino también comprender cómo se comporta a medida que la entrada de datos crece. A continuación, se detalla el análisis teórico de la complejidad y su validación empírica mediante pruebas de escalabilidad.

5.1. Análisis Teórico de la Complejidad

El sistema se compone de dos fases principales, cada una con su propio perfil de complejidad:

1. Generación de Waves (Algoritmo Greedy): Dado que el problema de partición óptima es NP-Hard, utilizamos un enfoque voraz. La complejidad temporal de ordenar los pedidos por prioridad es O(P log P) (donde P es la cantidad de pedidos). Al iterar sobre los pedidos y buscar el picker con menor carga, el peor caso es O(P * K), siendo K la cantidad de pickers. Sin embargo, gracias a la naturaleza de nuestro algoritmo, el tiempo de ejecución es altamente eficiente y determinista.
2. Búsqueda de Caminos (Dijkstra + Memoización): El algoritmo de Dijkstra tiene una complejidad temporal de O((V + E) log V), donde V son las celdas transitables y E las conexiones. La complejidad espacial del grafo es O(V + E). Sin embargo, la inclusión del Cache de Distancias (Memoización) cambia las reglas del juego: la primera consulta desde un nodo cuesta el tiempo de Dijkstra, pero todas las consultas posteriores desde

ese mismo nodo se resuelven en O(1), amortiguando enormemente el costo temporal del sistema completo.

5.2. Escalabilidad en Escenarios Sintéticos

Para evaluar la escalabilidad, se generaron almacenes automáticos de distintos tamaños, aumentando proporcionalmente la cantidad de pasillos, productos y pedidos.

Tabla 5.1: Escalabilidad en Escenarios Sintéticos Generados

Escenario	Dimensiones	Productos	Pasos (Ticks)	Tiempo (CPU)	Elementos Procesados	Faltantes
Chico	20x20	36	119	0.076 s	36/36 (100%)	0
Mediano	40x30	122	264	0.259 s	60/122 (49%)	22
Grande	60x40	247	792	0.784 s	60/247 (24%)	57
Enorme	80x50	407	775	1.542 s	60/407 (15%)	111

Análisis del crecimiento temporal: El tiempo de simulación escala de forma controlada (de 76 milisegundos a 1.5 segundos en el escenario más grande). Esto demuestra que el uso de listas de adyacencia y la memoización evitan una explosión combinatoria, manteniendo el sistema en rangos de tiempo aceptables.

Análisis de los Faltantes (Robustez): El alto número de productos faltantes en los escenarios Medianos, Grandes y Enormes no representa un fallo del algoritmo, sino una limitación del generador aleatorio de escenarios, el cual crea topologías con pasillos de sentido único que aíslan físicamente ciertos productos. Frente a esto, el sistema demostró robustez: en lugar de colapsar o lanzar una excepción fatal, el motor detectó la inalcanzabilidad topológica, la registró como "faltante" y continuó procesando el resto de los pedidos exitosamente.

5.3. Pruebas de Estrés en Escenarios Manuales y el Costo del Multipicker

Para probar el motor con topologías perfectas y alta densidad de pickers, se corrieron escenarios manuales de gran escala (con más de 1500 celdas transitables).

Tabla 5.2: Pruebas en Escenarios Manuales Complejos

Escenario	Pasos (Ticks)	Tiempo (CPU)	Elementos Procesados	Distancia Total	Faltantes
escenario_plus	387	8.383 s	167/167 (100%)	2407	0
escenario_grande	1519	8.719 s	354/359 (98.6%)	3733	2
manual_113952	369	15.377 s	354/359 (98.6%)	3765	2

El "Trade-off" del Multipicker Dinámico:
Se observa un aumento considerable en el tiempo de ejecución (de 8 a 15 segundos) en comparación con los escenarios sintéticos. Este incremento es el costo esperado de la simulación dinámica en tiempo real.
En estos escenarios, múltiples pickers transitan simultáneamente por el almacén. Cuando dos pickers detectan una colisión frontal o compiten por la misma celda en su próximo paso, el motor de simulación frena a uno de ellos y ejecuta una búsqueda de ruta alternativa (Dijkstra) en vivo para esquivar al compañero. Este proceso se repite miles de veces en los 1500 pasos de simulación.

- El costo computacional: 15 segundos para procesar más de 1500 ticks de simulación (aprox. 10 milisegundos por paso).
- El beneficio operativo: A cambio de ese tiempo de procesamiento, se logra un 98.6% de efectividad (recogiendo 354 de 359 productos) garantizando que los pickers nunca se atraviesen entre sí, resolviendo conflictos de tráfico en tiempo real.

5.4. El Impacto de la Memoización (Programación Dinámica)

La conclusión más contundente del análisis de complejidad es el impacto del CacheDistancias. Sin la implementación de esta técnica de memoización, las pruebas en los escenarios manuales (que tardan 15 segundos) habrían demorado horas. El motor de simulación consulta distancias miles de veces por segundo para resolver colisiones y planificar los siguientes movimientos. Al reducir esas consultas de un costo $O((V+E) \log V)$ a un costo constante $O(1)$, la memoización es lo que hace que el sistema pase de ser teóricamente inviable a ser operativamente eficiente.

6. Comparación de Estrategias y Trade-offs

Para determinar la calidad de los recorridos y el impacto en el rendimiento, se implementaron y compararon dos estrategias de ruteo:

- Estrategia A (Orden de Pedidos): Visita las ubicaciones en el orden estricto en que aparecen en los pedidos, sin importar la distancia geográfica entre ellas.
- Estrategia B (Vecino más Cercano): Aplica una heurística Greedy que, desde la posición actual, busca el producto pendiente más cercano y se dirige hacia él.

6.1. Resultados Experimentales

Se ejecutó la comparación en tres escenarios con topologías de distinta complejidad. Las métricas obtenidas se resumen en la siguiente tabla:
Tabla 6.1: Comparación de Estrategias A vs B por escenario



Resultados de la Simulación por Escenario

Comparativa de rendimiento entre estrategias A y B

Escenario	Estrategia	Tiempo (ms)	Distancia Total	Nodos Visitados	Ganador Distancia	Ganador Tiempo
Chico	A (Orden)	11.083	45.0	927	Empate	B
	B (Cercanía)	8.017	45.0	927		
Mediano	A (Orden)	124.933	331.0	11,664	B	B
	B (Cercanía)	62.778	250.0	11,664		
Grande	A (Orden)	509.465	1143.0	74,921	B	B
	B (Cercanía)	463.196	730.0	74,921		

6.2. Análisis de los Resultados

A partir de los datos obtenidos, se pueden extraer las siguientes conclusiones:

1. Diferencias entre estrategias: La Estrategia A sigue un patrón lineal y ciego, lo que puede generar zigzagueos severos si el flujo de pedidos no coincide con la distribución física del almacén. La Estrategia B es adaptativa, optimizando la ubicación espacial en cada paso.
2. Menor Distancia Recorrida: La Estrategia B presenta consistentemente la mejor distancia (o empata en escenarios minúsculos). En el escenario grande, logró recorrer 730 celdas frente a las 1143 de la Estrategia A, lo que representa un ahorro del 36% en el desgaste físico del picker.
3. Menor Tiempo de Ejecución: En contra de la creencia común de que "calcular el vecino más cercano" hace más lento al sistema, la Estrategia B fue más rápida en todos los escenarios medidos. Esto ocurre porque, al recorrer rutas más cortas (menos aristas), el motor de simulación tiene que calcular y renderizar una menor cantidad de pasos totales (ticks), compensando ampliamente el tiempo extra de CPU que requiere evaluar los candidatos.

4. ¿Existe una estrategia mejor en todos los escenarios? En términos de distancia, la Estrategia B domina casi por completo. En términos de tiempo, también fue superior en estas pruebas.

Si bien los resultados muestran a B como clara ganadora, el Trade-off (compromiso) algorítmico subyacente sigue existiendo:

- La Estrategia A realiza menos consultas al Caché de Distancias (complejidad algorítmica más baja O(K)
- La Estrategia B realiza múltiples consultas al caché para comparar vecinos (complejidad algorítmica más alta O(K^ 2)

En un escenario donde los pickers estuvieran estáticos (sin simulación de movimiento), la Estrategia A calcularía el plan de ruteo en menos milisegundos. Sin embargo, en un entorno dinámico (que es lo que simula nuestro proyecto), el costo de mover un picker ineficientemente por 400 celdas extra supera por completo el tiempo de cómputo de la Estrategia B.

6.3. Conclusión del Análisis

Para un sistema de Wave Picking real, la Estrategia B (Vecino más Cercano) es la opción indiscutiblemente superior. La evidencia empírica demuestra que priorizar la reducción de la distancia física no solo ahorra tiempo operativo humano y desgaste de los operarios, sino que también optimiza el tiempo de simulación del propio software al reducir la cantidad de eventos de movimiento a procesar.

La Estrategia A queda relegada a escenarios donde el orden de los pedidos sea estrictamente inamovible por reglas de negocio, o como un mecanismo de control para validar el ahorro de la Estrategia B.

7. Conclusiones y Trabajo Futuro

El sistema desarrollado cumple con todos los requisitos exigidos, demostrando que es posible modelar un problema logístico complejo aplicando estructuras de datos fundamentales (Grafos, HashTables, Heaps) y algoritmos deterministas.

Para implementar este sistema en un Warehouse real, se requerirían las siguientes modificaciones:

1. Integración en tiempo real: Conectar el sistema a una base de datos en la nube para actualizar el stock dinámicamente.
2. Manejo de imprevistos: Incorporar sensores IoT para detectar obstrucciones físicas (ej. un pallet caído) y recalcular rutas en vivo (Dijkstra dinámico).
3. Interfaz para operarios: Adaptar la aplicación a pantallas táctiles montadas en los carritos de picking o integrarla con escáneres de código de barras.



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



FACULTAD DE INGENIERIA